



Notebook - Maratonas de Programação

Prisioneiras de WA e WAstros

Contents	
1 Algoritmos	2
1.1 Submask	2
1.2 Bitwise	2
1.3 Ternary Search	2
2 Geometria	2
2.1 Reta Dois	2
2.2 Triangulos	2
2.3 Poligonos	4
2.4 Quadrilateros	5
2.5 3d	5
2.6 Retas	6
2.7 Circulos	7
2.8 Envoltorio Convexo	8
2.9 Sweep Line	9
2.10 Vetores	11
2.11 Retas Algoritmos	12
3 Strings	13
3.1 Rabinkarp	13
3.2 Trie	13
3.3 Elementar	14
3.4 Kmp	14
3.5 Busca	15
3.6 Cpp	15
3.7 Lcsubseq	15
3.8 Z-function	16
3.9 Notes	16
3.10 Edit-func	16
3.11 Hashes	17
3.12 Anagram Pos	18
4 Matematica	19
4.1 Fast Exp	19
4.2 Mdc	19
4.3 Primos	19
5 Grafos	20
5.1 Prim	20
5.2 Kruskal	20
5.3 Floyd Warshall	20
5.4 Tree Diameter	21
5.5 Bfs 01	21
5.6 Succ	21
5.7 Kosaraju	21
5.8 Dijkstra	21
5.9 Ford	22
6 DP	22
6.1 Kadane	22
6.2 Maxrs-ps	22
6.3 Tsp	22

6.4	Maxssm	22
6.5	Max2drs-k	23
6.6	Lis	23

1 Algoritmos

1.1 Submask

```
// O(3^n)
for (int m = 0; m < (1<<n); m++) {
    for (int s = m; s; s = (s-1) & m) {
        // s is every submask of m
    }
}

// O(2^n * n) SOS dp like
for (int b = n-1; b >= 0; b--) {
    for (int m = 0; m < (1 << n); m++) {
        if (j & (1 << b)) {
            // propagate info through submasks
            amount[j ^ (1 << b)] += amount[j];
        }
    }
}
```

1.2 Bitwise

```
// Least significant bit (lsb)
int lsb(int x) { return x&-x; }
int lsb(int x) { return __builtin_ctz(x); } // bit position

// Most significant bit (msb)
int msb(int x) { return 32-1-__builtin_clz(x); } // bit position

// Power of two
bool isPowerOfTwo(int x){ return x && (!(x&(x-1))); }
```

1.3 Ternary Search

```
// Ternary
ld l = -1e4, r = 1e4;
int iter = 100;
while(iter--){
    ld m1 = (2*l + r) / 3;
    ld m2 = (l + 2*r) / 3;
    if(check(m1) > check(m2))
        l = m1;
    else
        r = m2;
}
```

2 Geometria

2.1 Reta Dois

```
template<typename T>
struct Line {
    bool vertical;
    T m, b;
```

```
Line(const Point<T>& P, const Point<T>& Q) : vertical(false)
{
    if (equals(P.x, Q.x))
    {
        vertical = true;
        b = P.x;
    } else
    {
        m = (Q.y - P.y)/(Q.x - P.x)
        b = P.y - m * P.x
    }
}

bool operator==(const Line<T>& r) const // Verdadeiro se coincidentes
{
    if (vertical != r.vertical || !equals(m, r.m)) return false;

    return equals(b, r.b);
}

bool parallel(const Line<T>& r) const // Verdadeiro se paralelas
{
    if (vertical && r.vertical) return b != r.b;
    if (vertical || r.vertical) return false;

    return equals(m, r.m) && !equals(b, r.b);
}

bool orthogonal(const Line& r) const // Verdadeiro se perpendiculares
{
    if (vertical && r.vertical)
        return false;

    if ((vertical && equals(r.m, 0)) || (equals(m, 0) && r.vertical))
        return true;

    if (vertical || r.vertical)
        return false;

    return equals(m * r.m, -1.0);
}
};
```

2.2 Triangulos

```
template<typename T>
struct Triangle {
    Point<T> A, B, C;

    enum Angles { RIGHT, ACUTE, OBTUSE };
    enum Sides { EQUILATERAL, ISOSCELES, SCALENE };

    double perimeter() const
    {
        auto a = dist(A, B), b = dist(B, C), c = dist(C, A);

        return a + b + c;
    }
}
```

```

double area() const
{
    Line<T> r(A, B);

    auto b = dist(A, B);
    auto h = r.distance(C);

    return (b * h)/2;
}

double area2() const
{
    auto a = dist(A, B);
    auto b = dist(B, C);
    auto c = dist(C, A);
    auto s = (a + b + c)/2
    return sqrt(s)*sqrt(s - a)*sqrt(s - b)*sqrt(s - c);
}

double area3() const
{
    double det = (A.x*B.y + A.y*C.x + B.x*C.y) - (C.x*B.y + C.y*A.x + B.x*
A.y);

    return 0.5 * fabs(det);
}

Point<T> barycenter() const
{
    auto x = (A.x + B.x + C.x) / 3.0;
    auto y = (A.y + B.y + C.y) / 3.0;

    return Point<T> { x, y };
}

double inradius() const
{
    return (2 * area()) / perimeter();
}

Point<double> incenter() const
{
    auto a = dist(B, C), b = dist(A, C), c = dist(A, B);
    auto P = perimeter();
    auto x = (a*A.x + b*B.x + c*C.x)/P;
    auto y = (a*A.y + b*B.y + c*C.y)/P;

    return { x, y };
}

double circumradius() const
{
    auto a = dist(B, C);
    auto b = dist(A, C);
    auto c = dist(A, B);

    return (a * b * c)/(4 * area());
}

```

```

}

Point<T> circumcenter() const
{
    auto D = 2*(A.x*(B.y - C.y) + B.x*(C.y - A.y) + C.x*(A.y - B.y));

    auto A2 = A.x*A.x + A.y*A.y;
    auto B2 = B.x*B.x + B.y*B.y;
    auto C2 = C.x*C.x + C.y*C.y;

    auto x = (A2*(B.y - C.y) + B2*(C.y - A.y) + C2*(A.y - B.y))/D;
    auto y = (A2*(C.x - B.x) + B2*(A.x - C.x) + C2*(B.x - A.x))/D;

    return { x, y };
}

Point<T> orthocenter() const
{
    Line<T> r(A, B), s(A, C);

    Line<T> u { r.b, -r.a, -(C.x*r.b - C.y*r.a) };
    Line<T> v { s.b, -s.a, -(B.x*s.b - B.y*s.a) };

    auto det = u.a * v.b - u.b * v.a;
    auto x = (-u.c * v.b + v.c * u.b) / det;
    auto y = (-v.c * u.a + u.c * v.a) / det;

    return { x, y };
}

Angles classification_by_angles() const
{
    auto a = dist(A, B);
    auto b = dist(B, C);
    auto c = dist(C, A);

    auto alpha = acos((a*a - b*b - c*c)/(-2*b*c));
    auto beta = acos((b*b - a*a - c*c)/(-2*a*c));
    auto gamma = acos((c*c - a*a - b*b)/(-2*a*b));

    auto right = PI / 2.0;

    if (equals(alpha, right) || equals(beta, right) || equals(gamma, right
))
        return RIGHT;

    if (alpha > right || beta > right || gamma > right)
        return OBTUSE;

    return ACUTE;
}

Sides classification_by_sides() const
{
    auto a = dist(A, B), b = dist(B, C), c = dist(C, A);

    if (equals(a, b) and equals(b, c))
        return EQUILATERAL;
}

```

```

        if (equals(a, b) or equals(a, c) or equals(b, c))
            return ISOSCELES;

        return SCALENE;
    }
};

```

2.3 Poligonos

```

template<typename T>
class Polygon {
    vector<Point<T>> vs;
    int n;

    // 0 âmetro deve conter os n évertices do ípolgono
    Polygon(const vector<Point<T>>& ps) : vs(ps), n(vs.size())
    {
        vs.push_back(vs.front());
    }

    T D(const Point<T>& P, const Point<T>& Q, const Point<T>& R) const
    {
        return (P.x * Q.y + P.y * R.x + Q.x * R.y) - (R.x * Q.y + R.y * P.x +
        Q.x * P.y);
    }

    bool convex() const {
        // Um ípolgono deve ter, no mínimo, 3 évertices
        if (n < 3) return false;

        int P = 0, N = 0, Z = 0;

        for (int i = 0; i < n; ++i) {
            auto d = D(vs[i], vs[(i + 1) % n], vs[(i + 2) % n]);
            d ? (d > 0 ? ++P : ++N) : ++Z;
        }

        return P == n or N == n;
    }

    double distance(const Point<T>&P, const Point<T>& Q)
    {
        return hypot(P.x - Q.x, P.y - Q.y);
    }

    double perimeter() const
    {
        auto p = 0.0;
        for (int i = 0; i < n; ++i)
            p += distance(vs[i], vs[i + 1]);
        return p;
    }

    double area() const
    {
        auto a = 0.0;
        for (int i = 0; i < n; ++i)

```

```

{
    a += vs[i].x * vs[i + 1].y;
    a -= vs[i + 1].x * vs[i].y;
}
return 0.5 * fabs(a);
}

// Ângulo APB, em radianos
double angle(const Point<T>& P, const Point<T>& A, const Point<T>& B)
{
    auto ux = P.x - A.x;
    auto uy = P.y - A.y;
    auto vx = P.x - B.x;
    auto vy = P.y - B.y;
    auto num = ux * vx + uy * vy;
    auto den = hypot(ux, uy) * hypot(vx, vy);
    // Caso especial: se den == 0, algum dos vetores é degenerado: os
    // dois pontos ão iguais. Neste caso, o ângulo ão áest definido
    return acos(num / den);
}

bool equals(double x, double y) {
    static const double EPS { 1e-6 };
    return fabs(x - y) < EPS;
}

bool contains(const Point<T>& P) const
{
    if (n < 3) return false;
    auto sum = 0.0;
    for (int i = 0; i < n - 1; ++i) {
        auto d = D(P, vs[i], vs[i + 1]);
        auto a = angle(P, vs[i], vs[i + 1]);
        sum += d > 0 ? a : (d < 0 ? -a : 0);
    }
    static const double PI = acos(-1.0);
    return equals(fabs(sum), 2*PI);
}

// çãInterseo entre a reta AB e o segmento de reta PQ
Point<T> intersection(const Point<T>& P, const Point<T>& Q,
                     const Point<T>& A, const Point<T>& B)
{
    auto a = B.y - A.y;
    auto b = A.x - B.x;
    auto c = B.x * A.y - A.x * B.y;
    auto u = fabs(a * P.x + b * P.y + c);
    auto v = fabs(a * Q.x + b * Q.y + c);

    // éMdia ponderada pelas âdistncias de P e Q éat a reta AB
    return {(P.x * v + Q.x * u)/(u + v), (P.y * v + Q.y * u)/(u + v)};
}

// Corta o ípolgono com a reta r que passa por A e B
Polygon cut_polygon(const Point<T>& A, const Point<T>& B) const
{
    vector<Point<T>> points;
    const double EPS { 1e-6 };

```

```

for (int i = 0; i < n; ++i)
{
    auto d1 = D(A, B, vs[i]);
    auto d2 = D(A, B, vs[i + 1]);

    // é Vrtice à esquerda da reta
    if (d1 > -EPS)
        points.push_back(vs[i]);

    // A aresta cruza a reta
    if (d1 * d2 < -EPS)
        points.push_back(intersection(vs[i], vs[i + 1], A, B));
}

return Polygon(points);
}

double circumradius() const
{
    auto s = distance(vs[0], vs[1]);
    const double PI { acos(-1.0) };
    return (s/2.0)*(1.0/sin(PI/n));
}

double apothem() const
{
    auto s = distance(vs[0], vs[1]);
    const double PI { acos(-1.0) };
    return (s/2.0)*(1.0/tan(PI/n));
}
};

```

2.4 Quadriláteros

```

struct Rectangle {
    Point<T> P, Q;
    T b, h;

    Rectangle(const Point<T>& p, const Point<T>& q) : P(p), Q(q)
    {
        b = max(P.x, Q.x) - min(P.x, Q.x);
        h = max(P.y, Q.y) - min(P.y, Q.y);
    }

    Rectangle(const T& base, const T& height)
        : P(0, 0), Q(base, height), b(base), h(height) {}

    T perimeter() const
    {
        return 2 * (b + h);
    }

    T area() const
    {
        return b * h;
    }

    Rectangle intersection(const Rectangle& r) const
    {

```

```

        using interval = pair<T, T>;

        auto I = interval(min(P.x, Q.x), max(P.x, Q.x));
        auto U = interval(min(r.P.x, r.Q.x), max(r.P.x, r.Q.x));

        auto a = max(I.first, U.first);
        auto b = min(I.second, U.second);

        if (b < a)
            return { {-1, -1}, {-1, -1} };

        I = interval(min(P.y, Q.y), max(P.y, Q.y));
        U = interval(min(r.P.y, r.Q.y), max(r.P.y, r.Q.y));

        auto c = max(I.first, U.first);
        auto d = min(I.second, U.second);

        if (d < c)
            return { {-1, -1}, {-1, -1} };

        inter = Rectangle(Point(a, c), Point(b, d));

        return { {a, c}, {b, d} };
    }
};

template<typename T>
struct Trapezium {
    T b, B, h;

    T area() const
    {
        return (b + B) * h / 2;
    }
};

2.5 3d

template<typename T>
struct Point3D { T x, y, z; };

template<typename T>
struct Sphere {
    Point3D<T> C;
    T r;

    double area() const
    {
        return 4.0*PI*r*r;
    }

    double volume() const
    {
        return 4.0*PI*r*r*r/3.0;
    }
};

template<typename T>

```

```

struct Cylinder {
    T r, h;

    double area() const
    {
        return 2*PI*r*(r + h);
    }

    double volume() const
    {
        return PI*r*r*h;
    }
};

template<typename T>
struct Cube {
    T L;

    double face_diagonal() const
    {
        return L*sqrt(2.0);
    }

    double space_diagonal() const
    {
        return L*sqrt(3.0);
    }

    double area() const
    {
        return 6.0*L*L;
    }

    double volume() const
    {
        return L*L*L;
    }
};

template<typename T>
struct Cone {
    T r, H;

    double volume() const
    {
        return PI*r*r*H/3.0;
    }

    double area() const
    {
        return PI*r*r + PI*r*sqrt(r*r + H*H);
    }

    // Volume do tronco do cone
    double frustum(double rm, double h) const
    {
        return PI*h*(r*r + r*rm + rm*rm)/3.0;
    }
};

```

```

};

template<typename T>
struct Parallelepiped {
    Vector3D<T> u, v, w;

    double volume() const
    {
        return fabs(u.x*v.y*w.z + u.y*v.z*w.x + u.z*v.x*w.y
            - (u.x*v.z*w.y + u.y*v.x*w.z + u.z*v.y*w.x));
    }

    double volume2() const
    {
        double a = u.lenght();
        double b = v.length();
        double c = w.length();

        double m = angle(u, v);
        double n = angle(u, w);
        double p = angle(v, w);

        return a*b*c*sqrt(1 + 2*cos(m)*cos(n)*cos(p)
            - cos(m)*cos(m) - cos(n)*cos(n) - cos(p)*cos(p));
    }

    double volume3() const
    {
        return fabs(dot_product(u, cross_product(v, w)));
    }

    double area() const
    {
        double uv = cross_product(u, v).length();
        double uw = cross_product(u, w).length();
        double vw = cross_product(v, w).length();

        return 2*(uv + uw + vw);
    }
};

```

2.6 Retas

```

#include <bits/stdc++.h>

template<typename T>
struct Point {
    T x = 0, y = 0;
};

template<typename T>
struct Line {
    T a, b, c;

    Line(const Point<T>& P, const Point<T>& Q)
        : a(P.y - Q.y), b(Q.x - P.x), c(P.x * Q.y - Q.x * P.y)
    {
    }
};

```

```

bool contains(const Point<T>& P) const
{
    return equals(a*P.x + b*P.y + c, 0);
}

bool operator==(const Line<T>& r) const
{
    auto k = a ? a : b;
    auto s = r.a ? r.a : r.b;

    return equals(a*s, r.a*k) && equals(b*s, r.b*k) && equals(c*s, r.c*k);
}

bool parallel(const Line<T>& r) const
{
    auto det = a*r.b - b*r.a;

    return det == 0 and !(*this == r);
}

double distance(const Point<T>& p) const // âDistncia de p à reta
{
    return fabs(a*p.x + b*p.y + c)/hypot(a, b);
}

Point<T> closest(const Point<T>& p) const // Ponto mais óprximo de p
{
    auto den = (a*a + b*b);

    auto x = (b*(b*p.x - a*p.y) - a*c)/den;
    auto y = (a*(-b*p.x + a*p.y) - b*c)/den;

    return Point<T> { x, y };
}

bool orthogonal(const Line& r) const // Verdadeiro se perpendiculares
{
    return equals(a * r.a + b * r.b, 0);
}
};

template<typename T>
T absolute_value(T x)
{
    if constexpr (std::is_floating_point_v<T>)
        return fabs(x);
    else
        return llabs(static_cast<long long>(x));
}

template<typename T>
double dist(const Point<T>& P, const Point<T>& Q) {
    return hypot(static_cast<double>(P.x - Q.x), static_cast<double>(P.y - Q.y));
}

template<typename T>

```

```

T dist2(const Point<T>& P, const Point<T>& Q) {
    return (P.x - Q.x)*(P.x - Q.x) + (P.y - Q.y)*(P.y - Q.y);
}

template<typename T>
T taxicab(const Point<T>& P, const Point<T>& Q) {
    return absolute_value(P.x - Q.x) + absolute_value(P.y - Q.y);
}

template<typename T>
T max_norm(const Point<T>& P, const Point<T>& Q) {
    return std::max(absolute_value(P.x - Q.x), absolute_value(P.y - Q.y));
}

int main()
{
    Point<int> P, Q { 2, 3 };

    std::cout << "Euclidiana: " << dist(P, Q) << '\n';
    std::cout << "Quadrado: " << dist2(P, Q) << '\n';
    std::cout << "Motorista de átxi: " << taxicab(P, Q) << '\n';
    std::cout << "Norma do ámximo: " << max_norm(P, Q) << '\n';

    return 0;
}

```

2.7 Círculos

```

#define PI 2*acos(0.0)

// A unidade de medida do angulo é radianos
template<typename T>
struct Circle {
    Point<T> C;
    T r;

    enum { IN, ON, OUT } PointPosition;

    double perimeter() const {
        return 2.0 * PI * r;
    }

    double area() const {
        return PI * r * r;
    }

    double arc(double theta) const {
        return theta * r;
    }

    double sector(double theta) const {
        return (theta * r * r)/2;
    }

    double chord(double theta) const {
        return 2 * r * sin(theta/2);
    }
}

```



```

double segment(double a) const {
    return ((a - sin(a))*r*r)/2.0;
}

PointPosition position(const Point& P) const
{
    auto d = dist(P, C);

    return equals(d, r) ? ON : (d < r ? IN : OUT);
}

// O código abaixo foi adaptado do livro Competitive Programming 3. A
// função retorna um dos
// círculos: o outro pode ser encontrado invertendo os âparmetros P e Q na
// chamada da função

static std::optional<Circle>
from_2_points_and_r(const Point<T>& P, const Point<T>& Q, T r)
{
    double d2 = (P.x - Q.x) * (P.x - Q.x) + (P.y - Q.y) * (P.y - Q.y);
    double det = r * r / d2 - 0.25;
    if (det < 0.0)
        return { };
    double h = sqrt(det);
    auto x = (P.x + Q.x) * 0.5 + (P.y - Q.y) * h;
    auto y = (P.y + Q.y) * 0.5 + (Q.x - P.x) * h;
    return Circle<T>{ Point<T>(x, y), r };
}

static std::experimental::optional<Circle>
from_3_points(const Point<T>& P, const Point<T>& Q, const Point<T>& R)
{
    auto a = 2*(Q.x - P.x);
    auto b = 2*(Q.y - P.y);
    auto c = 2*(R.x - P.x);
    auto d = 2*(R.y - P.y);
    auto det = a*d - b*c;
    // Pontos colineares
    if (equals(det, 0))
        return { };
    auto k1 = (Q.x*Q.x + Q.y*Q.y) - (P.x*P.x + P.y*P.y);
    auto k2 = (R.x*R.x + R.y*R.y) - (P.x*P.x + P.y*P.y);
    // Solução do sistema por Regra de Cramer
    auto cx = (k1*d - k2*b)/det;
    auto cy = (a*k2 - c*k1)/det;
    Point<T> C { cx, cy };
    auto r = distance(P, C);
    return Circle<T>(C, r);
}
};

template<typename T> std::variant<int, std::vector<Point<T>>>
intersection(const Circle<T>& c1, const Circle<T>& c2)
{
    double d = distance(c1.C, c2.C);
    if (d > c1.r + c2.r or d < fabs(c1.r - c2.r)) return 0;
    if (equals(d, 0.0) and equals(c1.r, c2.r)) return oo;
}

```

```

auto a = (c1.r * c1.r - c2.r * c2.r + d * d)/(2 * d);
auto h = sqrt(c1.r * c1.r - a * a);
auto x = c1.C.x + (a/d)*(c2.C.x - c1.C.x);
auto y = c1.C.y + (a/d)*(c2.C.y - c1.C.y);
auto P = Point<T> { x, y };
x = P.x + (h/d)*(c2.C.y - c1.C.y);
y = P.y - (h/d)*(c2.C.x - c1.C.x);
auto P1 = Point<T> { x, y };
x = P.x - (h/d)*(c2.C.y - c1.C.y);
y = P.y + (h/d)*(c2.C.x - c1.C.x);
auto P2 = Point<T> { x, y };
return P1 == P2 ? std::vector<Point<T>> { P1 } : std::vector<Point<T>> {
P1, P2 };
}

// Interseção entre o círculo c e a reta que passa por P e Q
template<typename T> std::vector<Point<T>>
intersection(const Circle<T>& c, const Point<T>& P, const Point<T>& Q)
{
    auto a = pow(Q.x - P.x, 2.0) + pow(Q.y - P.y, 2.0);
    auto b = 2*((Q.x - P.x) * (P.x - c.C.x) + (Q.y - P.y) * (P.y - c.C.y));
    auto d = pow(c.C.x, 2.0) + pow(c.C.y, 2.0) + pow(P.x, 2.0)
        + pow(P.y, 2.0) + 2*(c.C.x * P.x + c.C.y * P.y);
    auto D = b * b - 4 * a * d;
    if (D < 0)
        return { };
    else if (equals(D, 0))
    {
        auto u = -b/(2*a);
        auto x = P.x + u*(Q.x - P.x);
        auto y = P.y + u*(Q.y - P.y);
        return { Point { x, y } };
    }
    auto u = (-b + sqrt(D))/(2*a);
    auto x = P.x + u*(Q.x - P.x);
    auto y = P.y + u*(Q.y - P.y);
    auto P1 = Point { x, y };
    u = (-b - sqrt(D))/(2*a);
    x = P.x + u*(Q.x - P.x);
    y = P.y + u*(Q.y - P.y);
    auto P2 = Point { x, y };
    return { P1, P2 };
}
}

```

2.8 Envoltório Convexo

```

template<typename T>
class GrahamScan {
    static Point<T> pivot(vector<Point<T>>& P)
    {
        size_t idx = 0;
        for (size_t i = 1; i < P.size(); ++i)
            if (P[i].y < P[idx].y or (equals(P[i].y, P[idx].y) and P[i].x > P[
idx].x))
                idx = i;
        swap(P[0], P[idx]);
        return P[0];
    }
}

```

```

static void sort_by_angle(vector<Point<T>>& P)
{
    auto P0 = pivot(P);
    sort(P.begin() + 1, P.end(), [&](const Point<T>& A, const Point<T>& B)
    {
        // pontos colineares: escolhe-se o mais próximo do piv
        if (equals(D(P0, A, B), 0)) return A.distance(P0) < B.distance(P0)
;
        auto alfa = atan2(A.y - P0.y, A.x - P0.x);
        auto beta = atan2(B.y - P0.y, B.x - P0.x);
        return alfa < beta;
    });
}

static vector<Point<T>> convex_hull(const vector<Point<T>>& points)
{
    vector<Point<T>> P(points);
    auto N = P.size();
    // Corner case: com 3 vértices ou menos, P é o próprio convex hull
    if (N <= 3) return P;
    sort_by_angle(P);
    vector<Point<T>> ch;
    ch.push_back(P[N - 1]);
    ch.push_back(P[0]);
    ch.push_back(P[1]);
    size_t i = 2;
    while (i < N)
    {
        auto j = ch.size() - 1;
        if (D(ch[j - 1], ch[j], P[i]) > 0)
            ch.push_back(P[i++]);
        else
            ch.pop_back();
    }
    // O envoltório é um caminho fechado: o primeiro ponto é igual ao
    último
    return ch;
}

// Cadeia montona de Andrew
template<typename T>
vector<Point<T>> make_hull(const vector<Point<T>>& points, vector<Point<T>>&
hull)
{
    for (const auto& p : points)
    {
        auto size = hull.size();

        while (size >= 2 and D(hull[size - 2], hull[size - 1], p) <= 0)
        {
            hull.pop_back();
            size = hull.size();
        }

        hull.push_back(p);
    }
}

```

```

}

return hull;
}

template<typename T>
vector<Point<T>> monotone_chain(const vector<Point<T>>& points)
{
    vector<Point<T>> P(points);

    sort(P.begin(), P.end());

    vector<Point<T>> lower, upper;

    lower = make_hull(P, lower);

    reverse(P.begin(), P.end());

    upper = make_hull(P, upper);

    lower.pop_back();
    lower.insert(lower.end(), upper.begin(), upper.end());

    return lower;
}

```

2.9 Sweep Line

```

vector<int> max_intersection(const vector<pll>& is)
{
    vector<pll> es;

    for (size_t i = 0; i < is.size(); ++i)
    {
        auto [a, b] = is[i];

        es.emplace_back(a, i + 1); // Evento de início
        es.emplace_back(b, -(i + 1)); // Evento de fim
    }

    sort(es.begin(), es.end());

    set<int> active, max_set;

    for (const auto& [_, i] : es)
    {
        if (i > 0)
            active.emplace(i);
        else
            active.erase(-i);

        if (active.size() >= max_set.size())
            max_set = active;
    }

    return { max_set.begin(), max_set.end() };
}

```

```

// bf

template<typename T>
set<Point<T>> intersections(int N, const vector<Segment<T>>& segments)
{
    set<Point<T>> ans;

    for (int i = 0; i < N; ++i)
    {
        auto s = segments[i];

        for (int j = i + 1; j < N; ++j)
        {
            auto r = segments[j];
            auto P = s.intersection(r);

            if (P) ans.insert(P.value());
        }
    }

    return ans;
}

// Closest

pair<Point, Point> closest_pair(int N, vector<Point>& ps)
{
    sort(ps.begin(), ps.end());

    // Este código assume que N > 1
    auto d = dist(ps[0], ps[1]);
    auto closest = make_pair(ps[0], ps[1]);

    set<ii> S;
    S.insert(ii(ps[0].y, ps[0].x));
    S.insert(ii(ps[1].y, ps[1].x));

    for (int i = 2; i < N; ++i)
    {
        auto P = ps[i];
        auto it = S.lower_bound(Point(P.y - d, 0));

        while (it != S.end())
        {
            auto Q = Point(it->second, it->first);

            if (Q.x < P.x - d)
            {
                it = S.erase(it);
                continue;
            }

            if (Q.y > P.y + d)
                break;

            auto t = dist(P, Q);

            if (t < d)

```

```

{
    d = t;
    closest = make_pair(P, Q);
}

    ++it;
}

    S.insert(ii(P.y, P.x));
}

    return closest;
}

// Bentley Ottman

void add_neighbor_intersections(const Segment& s, const set<Segment>& sl,
set<Point>& ans, priority_queue<Event>& events)
{
    // TODO: garantir que a busca identifique unicamente o elemento s,
    // através do ajuste fino da âvarivel Segment::sweep_x
    auto it = sl.find(s);

    if (it != sl.begin())
    {
        auto L = *prev(it);
        auto P = s.intersection(L);

        if (P and ans.count(P.value()) == 0)
        {
            events.push(Event { P.value(), Event::INTERSECTION, s.idx } );
            ans.insert(P.value());
        }
    }

    if (next(it) != sl.end())
    {
        auto U = *next(it);
        auto P = s.intersection(U);

        if (P and ans.count(P.value()) == 0)
        {
            events.push(Event { P.value(), Event::INTERSECTION, s.idx } );
            ans.insert(P.value());
        }
    }
}

set<Point> bentley_ottman(vector<Segment>& segments)
{
    set<Point> ans;
    priority_queue<Event> events;

    for (size_t i = 0; i < segments.size(); ++i)
    {
        events.push(Event { segments[i].A, Event::OPEN, i });
        events.push(Event { segments[i].B, Event::CLOSE, i });
    }
}

```

```

set<Segment> sl;

while (not events.empty())
{
    auto e = events.top();
    events.pop();

    Segment::sweep_x = e.P.x;

    switch (e.type) {
    case Event::OPEN:
    {
        auto s = segments[e.i];
        sl.insert(s);

        add_neighbor_intersections(s, sl, ans, events);
    }
    break;

    case Event::CLOSE:
    {
        auto s = segments[e.i];
        auto it = sl.find(s); // TODO: aqui é tambm

        if (it != sl.begin() and it != sl.end())
        {
            auto L = *prev(it);
            auto U = *next(it);
            auto P = L.intersection(U);

            if (P and ans.count(P.value()) == 0)
                events.push( Event { P.value(), Event::INTERSECTION, L.idx
        } );
        }

        sl.erase(it);
    }
    break;

    default:
        auto r = segments[e.i];
        auto p = sl.equal_range(r);

        vector<Segment> range(p.first, p.second);

        // Remove os segmentos que se interceptam
        sl.erase(p.first, p.second);

        // Reinsere os segmentos
        Segment::sweep_x += 0.1;

        sl.insert(range.begin(), range.end());

        // Procura çõintersees com os novos vizinhos
        for (const auto& s : range)
            add_neighbor_intersections(s, sl, ans, events);
    }
}

```

```

    }

    return ans;
}

2.10 Vetores

template<typename T>
struct Vector
{
    T x = 0, y = 0;

    Vector(const Point<T>& A, const Point<T>& B)
        : x(B.x - A.x), y(B.y - A.y) {}
};

template<typename T>
Vector<T> normalize(const Vector<T>& v)
{
    auto len = v.length();
    return { v.x / len, v.y / len };
}

template<typename T>
Point<T> rotate(const Point<T>& P, T angle)
{
    auto x = cos(angle) * P.x - sin(angle) * P.y;
    auto y = sin(angle) * P.x + cos(angle) * P.y;

    return { x, y };
}

template<typename T>
Point<T> rotate2(const Point<T>& P, T angle, const Point<T>& C)
{
    auto Q = translate(P, -C.x, -C.y);
    Q = rotate(Q, angle);
    Q = translate(Q, C.x, C.y);

    return Q;
}

template<typename T>
Vector<T> scale(const Vector<T>& v, T sx, T sy)
{
    return { sx * v.x, sy * v.y };
}

template<typename T>
Point<T> translate(const Point<T>& P, T dx, T dy)
{
    return { P.x + dx, P.y + dy };
}

template<typename T>
Vector<T> cross_product(const Vector<T>& u, const Vector<T>& v)
{
    auto x = u.y*v.z - v.y*u.z;

```

```

    auto y = u.z*v.x - u.x*v.z;
    auto z = u.x*v.y - u.y*v.x;

    return { x, y, z };
}

template<typename T>
T dot_product(const Vector<T>& u, const Vector<T>& v)
{
    return u.x * v.x + u.y * v.y;
}

// O retorno está no intervalo [0, pi]
template<typename T>
double angle(const Vector<T>& u, const Vector<T>& v)
{
    auto lu = u.length();
    auto lv = v.length();
    auto prod = dot_product(u, v);

    return acos(prod/(lu * lv));
}

```

2.11 Retas Algoritmos

```

// Ângulo entre os segmentos de reta PQ e RS
template<typename T>
double angle(const Point<T>& P, const Point<T>& Q, const Point<T>& R, const
    Point<T>& S)
{
    auto ux = P.x - Q.x;
    auto uy = P.y - Q.y;

    auto vx = R.x - S.x;
    auto vy = R.y - S.y;

    auto num = ux * vx + uy * vy;
    auto den = hypot(ux, uy) * hypot(vx, vy);
    // Caso especial: se den == 0, algum dos vetores é degenerado: os dois
    // pontos são iguais. Neste caso, o ângulo não está definido
    return acos(num / den);
}

template<typename T>
Line<T> perpendicular_bisector(const Point<T>& P, const Point<T>& Q)
{
    auto a = 2*(Q.x - P.x);
    auto b = 2*(Q.y - P.y);
    auto c = (P.x * P.x + P.y * P.y) - (Q.x * Q.x + Q.y * Q.y);
    return { a, b, c };
}

struct Segment {
    Point<T> A, B;
    // Verifica se o ponto P da reta r que contém A e B pertence ao segmento
    bool contains(const Point<T>& P) const {
        return equals(A.x, B.x) ? min(A.y, B.y) <= P.y and P.y <= max(A.y, B.
y)

```

```

        : min(A.x, B.x) <= P.x and P.x <= max(A.x, B.x);
    }
    // Esta abordagem não exige que P esteja sobre a reta AB
    bool contains2(const Point<T>& P) const {
        double dAB = dist(A, B), dAP = dist(A, P), dPB = dist(P, B);
        return equals(dAP + dPB, dAB);
    }
    // Ponto mais próximo de P no segmento AB
    Point<T> closest(const Point<T>& P) {
        Line<T> r(A, B);
        auto Q = r.closest(P);
        if (this->contains(Q)) return Q;
        auto distA = P.distanceTo(A);
        auto distB = P.distanceTo(B);
        if (distA <= distB) return A;
        else return B;
    }

    bool intersect(const Segment<T>& s) const
    {
        auto d1 = D(A, B, s.A);
        auto d2 = D(A, B, s.B);
        if ((equals(d1, 0) && contains(s.A)) || (equals(d2, 0) && contains(s.B
))) return true;
        auto d3 = D(s.A, s.B, A);
        auto d4 = D(s.A, s.B, B);
        if ((equals(d3, 0) && s.contains(A)) || (equals(d4, 0) && s.contains(B
))) return true;
        return (d1 * d2 < 0) && (d3 * d4 < 0);
    }
}

// Verifica se o ponto P pertence ao segmento de reta AB
template<typename T>
bool contains(const Point<T>& A, const Point<T>& B, const Point<T>& P)
{
    // Verifica se P está na região retangular
    auto xmin = min(A.x, B.x);
    auto xmax = max(A.x, B.x);
    auto ymin = min(A.y, B.y);
    auto ymax = max(A.y, B.y);
    if (P.x < xmin || P.x > xmax || P.y < ymin || P.y > ymax)
        return false;
    // Verifica se P está no triângulo
    return equals((P.y - A.y)*(B.x - A.x), (P.x - A.x)*(B.y - A.y));
}

// D = 0: R pertence a reta PQ
// D > 0: R à esquerda da reta PQ
// D < 0: R à direita da reta PQ
template<typename T>
T D(const Point<T>& P, const Point<T>& Q, const Point<T>& R)
{
    return (P.x * Q.y + P.y * R.x + Q.x * R.y) - (R.x * Q.y + R.y * P.x + Q.x
* P.y);
}

template<typename T>

```

```
std::pair<int, Point<T>> intersections(const Line<T>& r, const Line<T>& s)
{
    auto det = r.a * s.b - r.b * s.a;
    if (equals(det, 0)) // Coincidentes ou paralelas
    { return { (r == s) ? oo : 0, {} }; }
    }else // Concorrentes
    {
        auto x = (-r.c * s.b + s.c * r.b) / det;
        auto y = (-s.c * r.a + r.c * s.a) / det;
        return { 1, { x, y } };
    }
}
```

3 Strings

3.1 Rabinkarp

```
int f(char c) { return c - 'a' + 1; }

int h(const std::string& s) {
    long long ans = 0, p = 31, q = 1000000007;

    for (auto it = s.rbegin(); it != s.rend(); ++it) {
        ans = (ans * p) % q;
        ans = (ans + f(*it)) % q;
    }
    return ans;
}

int rabin_karp(const std::string& s, const std::string& p) {
    int n = s.size(), m = p.size(), occ = 0, hp = h(p);

    for (int i = 0; i <= n - m; i++) {
        auto b = s.substr(i, m);
        occ += (h(b) == hp && b == p) ? 1 : 0;
    }
    return occ;
}
```

3.2 Trie

```
using Node = map<char, int>;
using Trie = vector<Node>;

Trie build_naive(const string& s){
    int root = 0, next = 0;
    Trie trie(1); // Instancia o ón raiz vazio
    for (int i = s.size() - 1; i >= 0; --i){
        string suffix = s.substr(i);
        int v = root;
        for (auto c : suffix){
            auto it = trie[v].find(c);
            if (it != trie[v].end()){
                v = it->second;
            } else{
                trie.push_back({ });

```

```
                trie[v][c] = ++next;
                v = next;
            }
        }
    }
    return trie;
}

ostream& operator<<(ostream& os, const Trie& trie){
    using cii = tuple<char, char, int, int>;
    queue<cii> q;
    q.push(cii('_', '-', 0, 0));
    int level = 0;
    while (not q.empty()){
        auto [c, p, n, l] = q.front();
        q.pop();
        if (l > level){
            os << '\n';
            level = l;
        }
        os << "[" << n << ", " << c << " (" << p << ")] ";
        if (c != '#')
            for (const auto& p : trie[n])
                q.push(cii(p.first, c, p.second, l + 1));
    }
    return os;
}

bool search(const Trie& trie, const string& s){
    int v = 0;
    for (auto c : s){
        auto it = trie[v].find(c);
        if (it == trie[v].end()) return false;
        v = it->second;
    }
    return true;
}

Trie build_naive_with_marker(const string& s){
    int root = 0, next = 0;
    Trie trie(1); // Instancia o ón raiz vazio
    for (int i = s.size() - 1; i >= 0; --i){
        string suffix = s.substr(i) + '#';
        int v = root;
        for (auto c : suffix){
            if (c == '#'){
                trie[v][c] = i;
                break;
            }
            auto it = trie[v].find(c);
            if (it != trie[v].end()){
                v = it->second;
            } else{
                trie.push_back({ });
                trie[v][c] = ++next;
                v = next;
            }
        }
    }
}
```

```

    }
    return trie;
}

vector<int> find(const Trie& trie, const string& s) {
    vector<int> is;
    int v = 0;
    for (auto c : s) {
        auto it = trie[v].find(c);
        if (it == trie[v].end()) return is;

        v = it->second;
    }
    queue<int> q;
    q.push(v);
    while (not q.empty()) {
        auto u = q.front();
        q.pop();
        for (auto [c, v] : trie[u])
            if (c == '#') is.push_back(v);
            else q.push(v);
    }
    return is;
}

size_t unique_substrings(const Trie& trie) {
    size_t count = 0;
    queue<int> q;
    q.push(0);
    while (not q.empty()) {
        auto u = q.front();
        q.pop();
        for (auto [c, v] : trie[u])
            if (c != '#') {
                ++count;
                q.push(v);
            }
    }
    return count;
}

int main() {
    string s;
    cin >> s;
    auto trie2 = build_naive_with_marker(s);
    cout << trie2 << '\n';
    string p;
    cin >> p;
    auto is = find(trie2, p);

    cout << "indexes: ";
    for (size_t i = 0; i < is.size(); ++i)
        cout << is[i] << (i + 1 == is.size() ? '\n' : ' ');
    cout << s << " tem " << unique_substrings(trie2) << " substrings distintas\n";

    return 0;
}

```

3.3 Elementar

```

/* Construção de anagramas */
bool is_anagram(const std::string& s, const std::string& t)
{
    std::string a(s), b(t);

    std::sort(a.begin(), a.end());
    std::sort(b.begin(), b.end());

    return a == b;
}

/* Construção de histogramas */
map<char, int> histogram(const std::string& s)
{
    std::map<char, int> h;

    for (auto c : s)
        ++h[c];

    return h;
}

/* Tokenização */
int main() {
    std::istringstream is("123.456.789-10");
    std::string token;

    getline(is, token, '.');
    std::cout << token << '\n';           // token = "123";

    getline(is, token, '.');
    std::cout << token << '\n';           // token = "456";

    getline(is, token, '-');
    std::cout << token << '\n';           // token = "789";

    getline(is, token);
    std::cout << token << '\n';           // token = "10";

    return 0;
}

```

3.4 Kmp

```

vector<int> strong_borders(const string& P)
{
    int m = P.size(), t = -1;
    vector<int> bs(m + 1, -1);

    for (int j = 1; j <= m; ++j)
    {
        while (t > -1 and P[t] != P[j - 1])
            t = bs[t];
    }
}

```

```

        ++t;
        bs[j] = (j == m or P[t] != P[j]) ? t : bs[t];
    }

    return bs;
}

int KMP(const string& S, const string& P)
{
    int n = S.size(), m = P.size(), i = 0, j = 0, occ = 0;

    vector<int> bs = strong_borders(P);

    while (i <= n - m)
    {
        while (j < m and P[j] == S[i + j])
            ++j;

        if (j == m) ++occ;

        int shift = j - bs[j];
        i += shift;
        j = max(0, j - shift);
    }

    return occ;
}

int main()
{
    string S = "abaabbabaabaaba", P = "abaaba";
    auto bs = strong_borders(P);

    cout << "i\tsubs\t\tborder\tshift\n";

    for (size_t i = 0; i <= P.size(); ++i)
    {
        cout << i << '\t' << P.substr(0, i) << '\t';

        if (i < 8)
            cout << '\t';

        cout << bs[i] << '\t' << i - bs[i] << '\n';
    }

    cout << '\n' << KMP(S, P) << '\n';

    return 0;
}

```

3.5 Busca

/ Contagem de ocorrências de P em S */*

```

int occurrences(const string& P, const string& S)
{
    int m = P.size();
    int n = S.size();
    int occ = 0;

```

```

    for (int i = 0; i < n - m + 1; ++i)
        occ += (P == S.substr(i, m) ? 1 : 0);

    return occ;
}

```

3.6 Cpp

/ Find P em S O(nm) */*

```

int main()
{
    string S = "bananada", P = "ana";

    auto x = S.find(P);           // x = 1

    x = S.find(P, 2);             // x = 3
    x = S.find(P, 4);             // x = npos

    x = S.rfind(P);               // x = 3

    return 0;
}

/* Find first and last em S O(nm) */
int main()
{
    string S { "exemplo" }, P { "abcde" }, Q { "xyz" };

    auto x = S.find_first_of(P);  // x = 0
    x = S.find_first_not_of(P);   // x = 1
    x = S.find_last_of(P);        // x = 2
    x = S.find_last_not_of(P);    // x = 6

    auto y = S.find_first_of(Q);  // y = 1
    y = S.find_first_not_of(Q);   // y = 0
    y = S.find_last_of(Q);        // y = 1
    y = S.find_last_not_of(Q);    // y = 6

    return 0;
}

/* função search() O(n+m) */
auto it = search(S.begin(), S.end(), P.begin(), P.end());

```

3.7 Lcsubseq

// Longest Common Subsequence

```

string lcs(string x, string y) {
    int n = x.size(), m = y.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));

    for (int i=0; i<=n; i++) {
        for (int j=0; j<=m; j++) {
            if (i == 0 or j == 0) continue;
            if (x[i-1] == y[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;

```


}

3.8 Z-function

/

3.9 Notes

/ *

3.10 Edit-func

CO

```

int m = s.size(), n = t.size();

for (int i = 0; i <= m; ++i)
    st[i][0] = i*c_r;

for (int j = 1; j <= n; ++j)
    st[0][j] = j*c_i;

for (int i = 1; i <= m; ++i)
    for (int j = 1; j <= n; ++j) {
        int insertion = st[i][j - 1] + c_i, deletion = st[i - 1][j] + c_r;
        int change = st[i - 1][j - 1] + c_s*(s[i - 1] == t[j - 1] ? 0 : 1);
        st[i][j] = min({ insertion, deletion, change });
    }

return st[m][n];
}

int main()
{
    string s { "TEP" }, t { "ACCEPTED" };

    cout << edit(s, t) << '\n';

    for (size_t i = 0; i <= s.size(); i++)
    {
        for (size_t j = 0; j <= t.size(); j++)
            printf("%4d", st[i][j]);

        printf("\n");
    }

    return 0;
}

```

3.11 Hashes

```

int f(char c) { return c - 'a' + 1; }

/* Hash simples */
int h(const string& s)
{
    int N = s.size();
    long long ans = 0, p = 31, q = 1'000'000'007;

    for (int i = N - 1; i >= 0; --i){
        ans = (ans * p) % q;
        ans = (ans + f(s[i])) % q;
    }
    return ans;
}

/* Hash duplo */
int hi(long long pi, long long qi, const string& s)
{
    int N = s.size();
    long long ans = 0;

```

```

    for (int i = N - 1; i >= 0; --i) {
        ans = (ans * pi) % qi;
        ans = (ans + f(s[i])) % qi;
    }
    return ans;
}

pair<int, int> h(const string& s)
{
    constexpr long long p1 = 31, q1 = 1'000'000'007;
    constexpr long long p2 = 29, q2 = 1'000'000'009;

    return { hi(p1, q1, s), hi(p2, q2, s) };
}

int main()
{
    string s;
    cin >> s;

    auto [h1, h2] = h(s);

    cout << "(" << h1 << ", " << h2 << ")\n";

    return 0;
}

/* Contagem de substrings distintas */
constexpr ll p = 31, q = 1'000'000'007;

int h(const string& s)
{
    int N = s.size();
    ll ans = 0;

    for (int i = N - 1; i >= 0; --i) {
        ans = (ans * p) % q;
        ans = (ans + f(s[i])) % q;
    }

    return ans;
}

vector<ll> prefixes(const string& s)
{
    int N = s.size();
    vector<ll> ps(N, 0);

    for (int i = 0; i < N; ++i)
        ps[i] = h(s.substr(0, i + 1));

    return ps;
}

ll fast_exp_mod(ll a, ll n)
{
    ll res = 1, base = a;

```

```

while (n)
{
    if (n & 1)
        res = (res * base) % q;

    base = (base * base) % q;
    n >>= 1;
}

return res;
}

vector<ll> inverses(ll N)
{
    vector<ll> is(N);
    ll base = 1;

    for (int i = 0; i < N; ++i) {
        is[i] = fast_exp_mod(base, q - 2);
        base = (base * p) % q;
    }

    return is;
}

int h(int i, int j, const vector<ll>& ps, const vector<ll>& is)
{
    auto diff = i ? ps[j] - ps[i - 1] : ps[j];
    diff = (diff * is[i]) % q;
    return (diff + q) % q;
}

int unique_substrings(const string& s)
{
    int N = s.size();
    set<ll> hs;
    auto ps = prefixes(s);
    auto is = inverses(s.size());

    for (int i = 0; i < N; ++i) {
        for (int j = i; j < N; ++j) {
            auto hij = h(i, j, ps, is);
            hs.insert(hij);
        }
    }

    return hs.size();
}

```

3.12 Anagram Pos

```

#include <bits/stdc++.h>
#define MAX 250
using namespace std;
using ll = long long;
using ull = unsigned long long;

// Fatorial usando çãprogramao âdinmica. O(N).

```

```

vector<ull> fatAns(MAX, 0);
ull fastFat(int n){
    if(fatAns[n]) return fatAns[n];
    fatAns[n] = n * fastFat(n-1);
    return fatAns[n];
}

// Calcular o total de çõcombinacoes sem çõrepeties.
ull totalPermutations(string s){
    ull ans = 1;
    map<char, int> h; // Histograma para dividir pelo fatorial dos repetidos.
    for(int i = 0; i < s.size(); ++i){
        ans *= (i+1);
        h[s[i]] ++;
    }
    for(auto i : h) ans /= fastFat(i.second);
    return ans;
}

// Dado a string, retorna a çãposio da mesma dentre todas as çõpermutaes.
ull anagramPosition(string s){
    // 'sorted s' o primeiro anagrama.
    string ss = s;
    sort(ss.begin(), ss.end());
    ull ans = 0;
    ull p = 0;
    // Remover da string original éat que forme a desejada.
    while(!ss.empty()){
        unsigned int i = 0;
        while(i < ss.size()){
            if(ss[i] == s[p]){
                // cout << ss[i] << " YES." << endl;
                break;
            }
            else {
                string tempstr = ss;
                tempstr.erase(tempstr.begin() + i);
                ans += totalPermutations(tempstr);
                // cout << ss[i] << " NO. Sum : " << totalPermutations(tempstr) << endl;
                char temp = ss[i];
                while(ss[i] == temp) i++;
            }
        }
        ss.erase(ss.begin()+i); p++;
    }
    return ans;
}

// Mostrar todas as çõpermutaes ípossveis, simplesmente para provar o resultado
.
void showEveryAnagram(string s){
    sort(s.begin(), s.end());
    map<string, bool> check;
    ull i = 0;
    do{
        if(!check[s]) cout << i << " " << s << endl , ++i;
    }
}

```

```

    }
    while(next_permutation(s.begin(), s.end()));
}

```

4 Matematica

4.1 Fast Exp

```

#include <bits/stdc++.h>

using namespace std;

long long fast_exp(long long a, int n)
{
    if (n == 1)
        return a;

    auto x = fast_exp(a, n / 2);

    return x * x * (n % 2 ? a : 1);
}

long long fast_exp_it(long long a, int n)
{
    long long res = 1, base = a;

    while (n)
    {
        if (n & 1)
            res *= base;

        base *= base;
        n >>= 1;
    }

    return res;
}

int main()
{
    long long a;
    int n;

    cin >> a >> n;

    cout << a << "^" << n << " = " << fast_exp(a, n) << '\n';

    return 0;
}

```

4.2 Mdc

```

#include <bits/stdc++.h>

using namespace std;

```

```

long long gcd(long long a, long long b)
{
    return b ? gcd(b, a % b) : a;
}

long long ext_gcd(long long a, long long b, long long& x, long long& y)
{
    if (b == 0)
    {
        x = 1;
        y = 0;
        return a;
    }

    long long x1, y1;
    long long d = ext_gcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - y1*(a/b);

    return d;
}

int main()
{
    long long a, b;
    cin >> a >> b;

    cout << "(" << a << ", " << b << ") = " << gcd(a, b) << '\n';

    long long x, y;
    auto d = ext_gcd(a, b, x, y);

    cout << d << " = (" << a << ")(" << x << ") + (" << b << ")(" << y << ")\n";

    return 0;
}

```

4.3 Primos

```

//(N ** fi de p) % p == 1 sempre
// sistema reduzido de íresduo é os diferentes restos que deixam (7 vai ter t
=6) - pega todos os restos
// únmeros coprimos - únmero que mdc entre eles é 1
// coprimos de 6 = 1,4,5

// TEOREMA DE FERMAT
// a^p é congruente a a(mod p) - a é inteiro e p é primo
//

// TEOREMA DE EULER
// a^fi de m é congruente a 1 mod m
// ós de primo o fi é -1
// fatora em primo e sabe que é -1
// fi de qualquer valor é = fi de primo 1 * fi de primo 2

```

```

// Fatoracao em primos
#define ll long long

ll phi(){

}

ll fatp(int x){
    map<int, int> m;

    for(int i = 2; i * i < x; i++){
        while(x%i == 0){
            x/=i;
            m[i]++;
        }

    }

}

// verificar se é primo
bool is_p(int n){
    if(n < 2)
        return false;

    if(n == 2)
        return true;

    if(n%2 == 0)
        return false;

    for(int i = 3; i * i <= n; i+=2){
        if(n%i == 0)
            return false;
    }
    return true;
}

// crivo
vector<long, long> primes(ll N){
    bitset<MAX> sieve;
    vector<long long> ps{2};
    sieve.set();

    for(ll i = 3; i<=N; i+=2){
        if(sieve[i]){
            ps.push_back(i);
            for(ll j = i * i; j<=N; j+=2*i){
                sieve[j] = false;
            }
        }
    }
    return ps;
}

```

5 Grafos

5.1 Prim

```

int prim(int u, size_t N){
    set<int> C; C.emplace(u);
    priority_queue<ii, vector<ii>, greater<ii>> pq;
    for (auto [v, w] : adj[u]) pq.emplace(w, v);
    int mst = 0;
    while (C.size() < N) {
        int v, w;
        do {
            tie(w, v) = pq.top(); pq.pop();
        } while (C.count(v));
        mst += w; C.emplace(v);
        for (auto [s, p] : adj[v]) pq.emplace(p, s);
    }
    return mst; }

```

5.2 Kruskal

```

class UnionFind {
    vector<int> parent, rank;

public:
    UnionFind(int N) : parent(N + 1), rank(N + 1, 0)
    { iota(parent.begin(), parent.end(), 0); }

    bool same_set(int i, int j) return find_set(i) == find_set(j);

    void union_set(int i, int j) {
        if (same_set(i, j)) return;
        int p = find_set(i);
        int q = find_set(j);
        if (rank[p] > rank[q]) parent[q] = p;
        else if (rank[q] > rank[p]) parent[p] = q;
        else { parent[q] = p; ++rank[p]; }
    }

    int find_set(int i) return parent[i] == i ? i : (parent[i] = find_set(
parent[i]));
};

int kruskal(int N, vector<edge>& es) {
    sort(es.begin(), es.end());
    int cost = 0; UnionFind ufs(N);
    for (auto [w, u, v] : es)
        if (not ufs.same_set(u, v)) {
            cost += w; ufs.union_set(u, v);
        }
    return cost; }

```

5.3 Floyd Warshall

```

// Floyd Warshall

int dist[N][N];

```

```

for(int k = 1; k <= n; k++)
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

```

5.4 Tree Diameter

```

pair<int, int> bfs(int s, int N) {
    vector<int> dist(N + 1, oo); dist[s] = 0;
    queue<int> q; q.push(s);
    int last = s;
    while (not q.empty()) {
        auto u = q.front(); q.pop();
        last = u;
        for (auto v : adj[u]) if (dist[v] == oo) { dist[v] = dist[u] + 1; q.
push(v); }
    }
    return { last, dist[last] }; }

int diameter(int N) {
    auto [v, _] = bfs(1, N);
    auto [w, D] = bfs(v, N); return D; }

```

5.5 Bfs 01

```

vector<int> d(n, INF);
deque<int> q;

void bfs(int x){
    d[x] = 0;
    q.push_front(x);
    while(!q.empty()){
        int u = q.front();
        q.pop_front();
        for(auto e: grafo[u]){
            int v = edge.ff;
            int w = edge.ss;
            if(d[v] > d[u] + w){
                d[v] = d[u] + w;
                if(w == 1)
                    q.push_back(v);
                else
                    q.push_front(v);
            }
        }
    }
}

```

5.6 Succ

```

void precomp(int N, int M, const vector<int>& s) {
    for (int u = 1; u <= N; ++u) S[u][0] = s[u];
    for (int i = 1; i <= M; ++i)
        for (int u = 1; u <= N; ++u)
            S[u][i] = S[S[u][i - 1]][i - 1];
}

```

```

int succ(int u, int k) {
    for (int i = 0; (1 << i) <= k; ++i) if (k & (1 << i)) u = S[u][i];
    return u;
}

```

5.7 Kosaraju

```

vector<int> g[N], gi[N]; // grafo invertido
int vis[N], comp[N]; // componente conexo de cada vertice
stack<int> S;

void dfs(int u){
    vis[u] = 1;
    for(auto v: g[u]) if(!vis[v]) dfs(v);
    S.push(u);
}

void scc(int u, int c){
    vis[u] = 1; comp[u] = c;
    for(auto v: gi[u]) if(!vis[v]) scc(v, c);
}

void kosaraju(int n){
    for(int i=0;i<n;i++) vis[i] = 0;
    for(int i=0;i<n;i++) if(!vis[i]) dfs(i);
    for(int i=0;i<n;i++) vis[i] = 0;
    while(S.size()){
        int u = S.top();
        S.pop();
        if(!vis[u]) scc(u, u);
    }
}

```

5.8 Dijkstra

```

#define pii pair<int, int>
vector<vector<pii>> g(N);
vector<bool> used(N);
vector<ll> d(N, LLINF);
priority_queue< pii, vector<pii>, greater<pii> > fila;

void dijkstra(int k) {
    d[k] = 0;
    fila.push({0, k});

    while (!fila.empty()) {
        auto [w, u] = fila.top();
        fila.pop();
        if (used[u]) continue;
        used[u] = true;

        for (auto [v, w]: g[u]) {
            if (d[v] > d[u] + w) {
                d[v] = d[u] + w;
                fila.push({d[v], v});
            }
        }
    }
}

```

```

    }
}
}

```

5.9 Ford

```

const int N = 2000010;

struct Ford {
    struct Edge {
        int to, f, c;
    };

    int vis[N];
    vector<int> adj[N];
    vector<Edge> edges;
    int cur = 0;

    void addEdge(int a, int b, int cap, int rcap) {
        Edge e;
        e.to = b; e.c = cap; e.f = 0;
        edges.pb(e);
        adj[a].pb(cur++);

        e = Edge();
        e.to = a; e.c = rcap; e.f = 0;
        edges.pb(e);
        adj[b].pb(cur++);
    }

    int dfs(int s, int t, int f, int tempo) {
        if(s == t)
            return f;
        vis[s] = tempo;

        for(int e : adj[s]) {
            if(vis[edges[e].to] < tempo and (edges[e].c - edges[e].f) > 0) {
                if(int a = dfs(edges[e].to, t, min(f, edges[e].c-edges[e].f),
tempo)) {
                    edges[e].f += a;
                    edges[e^1].f -= a;
                    return a;
                }
            }
        }
        return 0;
    }

    int flow(int s, int t) {
        int mflow = 0, tempo = 1;
        while(int a = dfs(s, t, INF, tempo)) {
            mflow += a;
            tempo++;
        }
        return mflow;
    }
};

```

6 DP

6.1 Kadane

```

int kadane(const vector<int>& as) {
    vector<int> s(as.size());
    s[0] = as[0];

    for (size_t i = 1; i < as.size(); ++i)
        s[i] = max(as[i], s[i - 1] + as[i]);

    return *max_element(s.begin(), s.end());
}

```

6.2 Maxrs-ps

```

long long MRS(const vector<int>& as)
{
    auto N = as.size() - 1;
    vector<long long> ps(N + 1, 0);
    // 0 código assume que os índices da sequência as começam em 1
    for (size_t i = 1; i <= N; ++i)
        ps[i] = ps[i - 1] + as[i];

    long long ans = -oo;
    for (size_t i = 1; i <= N; ++i)
        for (size_t j = i; j <= N; ++j)
            ans = max(ans, ps[j] - ps[i - 1]);

    return ans;
}

```

6.3 Tsp

```

int dist[MAX][MAX];
int st[MAX][1 << MAX];
int tsp(int i, int mask, int N) {
    if (mask == (1 << N) - 1) return dist[i][0];
    if (st[i][mask] != -1) return st[i][mask];
    int res = oo;
    for (int j = 0; j < N; ++j) {
        if (mask & (1 << j)) continue;

        res = min(res, tsp(j, mask | (1 << j), N) + dist[i][j]);
    }
    return (st[i][mask] = res);
}

```

6.4 Maxssm

```

int A[MAX][MAX], S[MAX][MAX];
int MSS(int N, int M) {
    for (int i = 0; i < N; ++i)
        S[i][0] = A[i][0];
    for (int j = 0; j < M; ++j)
        S[0][j] = A[0][j];
}

```

```

int ans = 0;
for (int i = 1; i < N; ++i)
    for (int j = 1; j < M; ++j) {
        S[i][j] = A[i][j] == 0 ? 0 : min({ S[i-1][j], S[i][j-1], S[i-1][j-1] }) + 1;

        ans = max(ans, S[i][j]);
    }
return ans;
}

```

6.5 Max2drs-k

```

int kadane(int N, const vector<int>& as) {
    vector<int> s(N + 1);
    s[1] = as[1];
    for (size_t i = 2; i < as.size(); ++i)
        s[i] = max(as[i], s[i - 1] + as[i]);
    return *max_element(s.begin() + 1, s.end());
}

int MSR(int N, int M, const vector<vector<int>>& A) {
    int ans = -oo;
    for (int i = 1; i <= M; ++i) {
        vector<int> r(N + 1, 0);

```

```

        for (int j = i; j <= M; ++j) {
            for (int k = 1; k <= N; ++k)
                r[k] += A[k][j];

            ans = max(ans, kadane(N, r));
        }
    }
return ans;
}

```

6.6 Lis

```

int LIS(int N, const vector<int>& as) {
    vector<int> lis(N + 1, oo);
    lis[0] = -oo;
    auto ans = 0;
    for (int i = 0; i < N; ++i) {
        auto it = lower_bound(lis.begin(), lis.end(), as[i]);
        auto pos = (int) (it - lis.begin());
        ans = max(ans, pos);
        lis[pos] = as[i];
    }
    return ans;
}

```